
Chapitre 6 : Décorateurs et Introspection

6.1. Fonctions de Première Classe et Closures

En Python, les fonctions sont des objets de première classe (*first-class citizens*) : elles peuvent être assignées à des variables, passées en arguments et retournées par d'autres fonctions.

Une **closure** (fermeture) se produit lorsqu'une fonction interne capture et se souvient des variables de son environnement englobant, même après l'exécution de la fonction externe.

```
def creer_multiplicateur(facteur: int):
    # La fonction englobée conserve l'accès à `facteur`
    def multiplier(nombre: int) -> int:
        return nombre * facteur

    return multiplier

double = creer_multiplicateur(2)
triple = creer_multiplicateur(3)

print(double(5)) # 10
print(triple(5)) # 15
```

6.2. Principes et Construction des Décorateurs

Un **décorateur** est un patron de conception (*Decorator Pattern*) appliqué aux fonctions ou aux classes. Il s'agit d'une fonction qui prend une autre fonction en argument pour en modifier ou étendre le comportement sans altérer son code source.

La syntaxe `@decorateur` n'est que du sucre syntaxique pour `fonction = decorateur(fonction)`.

```
import functools
import time

def mesurer_temps(func):
    @functools.wraps(func) # Preserve le nom et la docstring de `func`
    def wrapper(*args, **kwargs):
        debut = time.perf_counter()
        resultat = func(*args, **kwargs)
        fin = time.perf_counter()
        print(f"[{func.__name__}] Exécuté en {fin - debut:.6f} s")
        return resultat
```

```
    return wrapper

@mesurer_temps
def somme_carres(n: int) -> int:
    """Calcule la somme des carrés jusqu'à n."""
    return sum(i**2 for i in range(n))

somme_carres(1_000_000)
```

Note : L'utilisation de `functools.wraps` est essentielle pour préserver la métadonnée d'origine (`__name__`, `__doc__`) de la fonction décorée pour la réflexivité et l'introspection.

6.3. Décorateurs avec Arguments

Pour passer des arguments à un décorateur lui-même, il faut ajouter un niveau d'imbrication supplémentaire (un "générateur de décorateur").

```
import functools

def repeter(n_fois: int):
    def decorateur(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            for _ in range(n_fois):
                resultat = func(*args, **kwargs)
            return resultat
        return wrapper
    return decorateur

@repeter(n_fois=3)
def saluer(nom: str):
    print(f"Bonjour {nom}")

saluer("Alice")
```

Chapitre 7 : Itérateurs, Générateurs et Protocoles

7.1. Le Protocole d'Itération (`__iter__` et `__next__`)

Un **itérable** est un objet capable de retourner un **itérateur** via `iter(obj)` (qui appelle `obj.__iter__()`).

Un **itérateur** est un objet qui produit les éléments un par un via `next(obj)` (qui appelle `obj.__next__()`) et lève une exception `StopIteration` lorsqu'il n'y a plus d'éléments.

```
class CompteARebours:

    def __init__(self, depart: int):
        self.courant = depart

    def __iter__(self):
        return self

    def __next__(self):
        if self.courant <= 0:
            raise StopIteration
        valeur = self.courant
        self.courant -= 1
        return valeur

for n in CompteARebours(3):
    print(n)  # 3, 2, 1
```

7.2. Générateurs et Évaluation Paresseuse (*Lazy Evaluation*)

Un **générateur** simplifie l'écriture des itérateurs en utilisant le mot-clé `yield`. Au lieu de retourner une valeur et de détruire la pile d'exécution (comme `return`), `yield` gèle l'état de la fonction et produit une valeur à la volée.

- **Économie mémoire** : Les données ne sont pas chargées entièrement en mémoire RAM.

```
def lire_gros_fichier(chemin: str):
    with open(chemin, "r", encoding="utf-8") as f:
        for ligne in f:
            yield ligne.strip()

# Expression génératrice (analogue aux compréhensions, mais avec des
# parenthèses)
carres_gen = (x**2 for x in range(10_000_000))
print(next(carres_gen))  # 0
print(next(carres_gen))  # 1
```

Chapitre 8 : Gestionnaires de Contexte (*Context Managers*)

Les gestionnaires de contexte garantissent la gestion sécurisée des ressources (fichiers, connexions réseau, verrous) via l'instruction `with`, assurant leur nettoyage même en cas d'exception.

8.1. Le Protocole `__enter__` et `__exit__`

```
class ConnexionBD:

    def __init__(self, db_name: str):
        self.db_name = db_name

    def __enter__(self):
        print(f"Connexion à {self.db_name} établie.")
        return self # Valeur assignée au 'as' dans le bloc with

    def __exit__(self, exc_type, exc_val, exc_tb):
        print(f"Fermeture de la connexion {self.db_name}.")
        if exc_type is not None:
            print(f"Une erreur est survenue : {exc_val}")
        # Retourner True étoufferait l'exception (déconseillé sauf cas
        # spécifiques)
        return False

with ConnexionBD("production.db") as db:
    print("Exécution des requêtes SQL...")
```

8.2. Simplification avec `@contextmanager`

Le module `contextlib` permet de transformer une fonction génératrice en gestionnaire de contexte avec une syntaxe plus concise.

```
from contextlib import contextmanager

@contextmanager
def temps_execution(nom_tache: str):
    debut = time.perf_counter()
    try:
        yield
    finally:
        duree = time.perf_counter() - debut
        print(f"Tâche '{nom_tache}' exécutée en {duree:.4f}s")

with temps_execution("Calcul lourd"):
```

```
sum(i**2 for i in range(5_000_000))
```

Chapitre 9 : Typage Avancé, Métaclasses et Modèle Objet

9.1. Type Hinting Avancé et Duck Typing Statique (**Protocol**)

Introduit dans **typing**, le décorateur **@runtime_checkable** et les **Protocol** permettent le *Structural Subtyping* (Duck Typing vérifiable statiquement sans héritage direct).

```
from typing import Protocol, runtime_checkable

@runtime_checkable
class ExportableJSON(Protocol):

    def to_json(self) -> str:
        ...

class Produit:

    def to_json(self) -> str:
        return '{"nom": "Clavier"}'

def exporter(item: ExportableJSON) -> None:
    print(item.to_json())

p = Produit()
exporter(p)  # Valide statiquement (mypy) et à l'exécution avec
isinstance(p, ExportableJSON)
```

9.2. Les Métaclasses (**type**)

En Python, **tout est objet**, y compris les classes elles-mêmes. Une **métaclasse** est la classe d'une classe. Par défaut, la métaclasse de toutes les classes Python est **type**.

Les métaclasses permettent d'intercepter, modifier ou valider la création de classes au moment de leur définition (lors du chargement du module).

```
# Métaclasse qui force le nom de tous les attributs en majuscules ou lève
une erreur
class MetaControleSingleton(type):
    _instances = {}
```

```
def __call__(cls, *args, **kwargs):
    """Contrôle l'instanciation pour implémenter un Singleton au niveau
    Métaclasse."""
    if cls not in cls._instances:
        cls._instances[cls] = super().__call__(*args, **kwargs)
    return cls._instances[cls]

class Configuration(metaclass=MetaControleSingleton):

    def __init__(self):
        self.app_name = "MonApp"

cfg1 = Configuration()
cfg2 = Configuration()

print(cfg1 is cfg2)  # True (Même instance physique)
```

Chapitre 10 : Concurrency, Asynchronisme et Performance

10.1. Le GIL (Global Interpreter Lock)

Le GIL est un verrou dans CPython qui empêche plusieurs threads natifs d'exécuter du bytecode Python en parallèle sur plusieurs cœurs CPU.

- **Tâches orientées I/O (réseau, disque) :** Le GIL est libéré lors de la saisie/sortie → Utiliser `threading` ou `asyncio`.
- **Tâches orientées CPU (calculs mathématiques) :** Le GIL bloque l'exécution parallèle → Utiliser `multiprocessing`.

10.2. Threads vs Processus (`concurrent.futures`)

```
from concurrent.futures import ProcessPoolExecutor, ThreadPoolExecutor

def calcul_cpu(n: int) -> int:
    return sum(i**2 for i in range(n))

# Parallélisme réel sur plusieurs cœurs CPU
if __name__ == "__main__":
    avec_processus = [5_000_000, 5_000_000, 5_000_000]
    with ProcessPoolExecutor() as executor:
        resultats = list(executor.map(calcul_cpu, avec_processus))
```

```
print(resultats)
```

****10.3. Programation Asynchrone avec `asyncio`****

L'asynchronisme repose sur une **boucle d'événements** (*Event Loop*) mono-threadée capable de commuter entre des coroutines suspendues au niveau des points d'attente I/O (`await`).

- **`async def`** : Déclare une coroutine.
- **`await`** : Suspend la coroutine courante et rend la main à l'Event Loop.

```
import asyncio

async def telecharger_page(id_page: int, delai: int) -> str:
    print(f"Début téléchargement page {id_page}")
    await asyncio.sleep(delai) # Simule un appel réseau non-bloquant
    print(f"Fin téléchargement page {id_page}")
    return f"Contenu de la page {id_page}"

async def main():
    # Lancement en parallèle de plusieurs coroutines
    taches = [
        telecharger_page(1, 2),
        telecharger_page(2, 1),
        telecharger_page(3, 3),
    ]

    # asyncio.gather exécute les tâches simultanément
    resultats = await asyncio.gather(*taches)
    print(resultats)

# Exécution de la boucle d'événements
asyncio.run(main())
```